

Anish Ghosh
9/20/25
Dr. Ching-Hua Chuan
University of Miami

Image Classification: Facial Expression Recognition with CreateML

Introduction

In this project, I test the effect of training dataset size on the performance of a machine learning model for facial expression classification developed in CreateML. The experiment involves creating five datasets of increasing size and training five classification models on these datasets. I will then discuss the applications of these kinds of accessible machine learning tools and give an example application of the tool.

Data Preparation

For this experiment, I am working with the publicly available “facial_expressions” repository, which is a dataset of different facial expressions used for training machine learning algorithms uploaded by Brian Lee Yung Rowe. The data needs to be sorted into file folders for CreateML to easily access the images and their tags. I created a small Python script that would read each image name and create a copy of that image in a folder named after its emotional tag.

```
import os
import shutil
import pandas as pd

csv_path = "/facial_expressions-master/data/500_picts_satz.csv"
images_folder = "/facial_expressions-master/images"
output_base = "/Expressions"

df = pd.read_csv(csv_path)

for idx, row in df.iterrows():
    filename = row['image']    # column with image file name
    emotion = row['emotion']   # column with label

    # Build source and destination paths
    src = os.path.join(images_folder, filename)
    dest_folder = os.path.join(output_base, emotion)
```

```

dest = os.path.join(dest_folder, filename)

# Create destination folder if it doesn't exist
os.makedirs(dest_folder, exist_ok=True)

# Copy image if it exists
if os.path.exists(src):
    shutil.copy(src, dest)
else:
    print(f"File not found: {src}")

```

The script created a folder for each of the five emotions: anger, fear, happiness, neutral, and sadness. The largest training set will have 50 images in each category. If our test dataset has 20 images in each category, then there will be roughly a 70/30 split between the training and testing dataset sizes. The issue is that all categories have more than 70 images, except fear and sadness. I used the *legend.csv* data in the repository to add more labeled images to each category. And lastly, the repository provided a set of test images uploaded by students that were labeled in the title of each image. I manually added these images to each category. The resulting count for the fear category was 74, and sadness now has 181. Now, with at least 70 images in each category, I created a short Python script that would create 6 datasets: one test set (20 images per category) and five training sets: 10, 20, 30, 40, and 50 images per category

```

import os
import random
import shutil
from PIL import Image

base_dir = "/Expression Classifier/Expressions"
output_dir = "/Expression Classifier/Datasets"
random.seed(42) # reproducibility

# Detect emotion categories automatically
emotions = [d for d in os.listdir(base_dir) if
os.path.isdir(os.path.join(base_dir, d))]
print(f"Detected emotions: {emotions}")

def copy_as_jpg(src, dst):

```

```

ext = os.path.splitext(src)[1].lower()
if ext == ".png":
    img = Image.open(src).convert("RGB")
    dst = os.path.splitext(dst)[0] + ".jpg"
    img.save(dst, "JPEG")
else:
    shutil.copy(src, dst)

# Step 1: Create Test Set (20 per class)
test_dir = os.path.join(output_dir, "TestSet")
os.makedirs(test_dir, exist_ok=True)

remaining_files = {} # leftover files for training

for emotion in emotions:
    emotion_src = os.path.join(base_dir, emotion)
    emotion_dst = os.path.join(test_dir, emotion)
    os.makedirs(emotion_dst, exist_ok=True)

    files = [f for f in os.listdir(emotion_src) if
f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    if len(files) < 20:
        raise ValueError(f"Not enough images in {emotion}")

    chosen = random.sample(files, 20)

    for f in chosen:
        src_path = os.path.join(emotion_src, f)
        dst_path = os.path.join(emotion_dst, f)
        copy_as_jpg(src_path, dst_path)
        files.remove(f)

    remaining_files[emotion] = files

print(f"Test set created with {20 * len(emotions)} images")

# Step 2: Create Train Sets of 10,20,30,40,50 per class

```

```

dataset_sizes = [10, 20, 30, 40, 50]

for size in dataset_sizes:
    train_dir = os.path.join(output_dir, f"TrainSet_{size}")
    os.makedirs(train_dir, exist_ok=True)

    for emotion in emotions:
        emotion_src = os.path.join(base_dir, emotion)
        emotion_dst = os.path.join(train_dir, emotion)
        os.makedirs(emotion_dst, exist_ok=True)

        available = remaining_files[emotion]
        if len(available) < size:
            raise ValueError(f"Not enough images in {emotion} to
sample {size} for TrainSet_{size}")

        chosen = random.sample(available, size)

        for f in chosen:
            src_path = os.path.join(emotion_src, f)
            dst_path = os.path.join(emotion_dst, f)
            copy_as_jpg(src_path, dst_path)

    print(f"TrainSet_{size} with {size * len(emotions)} images")

```

While the ratio of training to test dataset size decreases as the training set increases, it is important to keep the test set the same across all builds to allow for comparison between the models. Additionally, while preparing the data, I noticed some of the labels did not match my interpretation of the emotion being expressed in the image. This may cause issues in the performance of the model and will be discussed in the applications and discussion section.

Building

For each model, I have provided an overview of the accuracy, precision, and recall.

Accuracy can be calculated as

$$Accuracy = \frac{\sum_{i=1}^c TP_i}{\sum_{i=1}^c (TP_i + FP_i + FN_i + TN_i)}$$

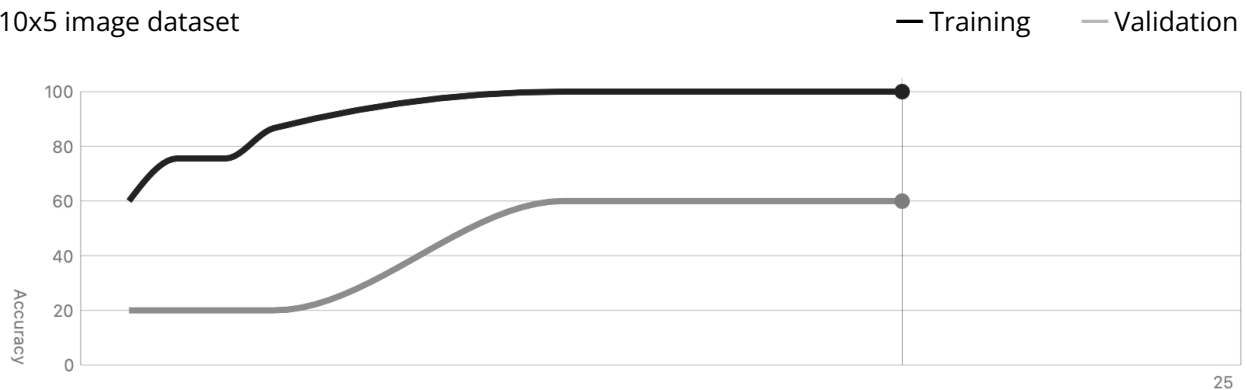
Precision can be calculated as

$$Precision_i = \frac{TP_i}{TP_i + FP_i}, Precision_{macro} = \frac{1}{c} \sum_{i=1}^c Precision_i$$

Recall can be calculated as

$$Recall_i = \frac{TP_i}{TP_i + FN_i}, Recall_{macro} = \frac{1}{c} \sum_{i=1}^c Recall_i$$

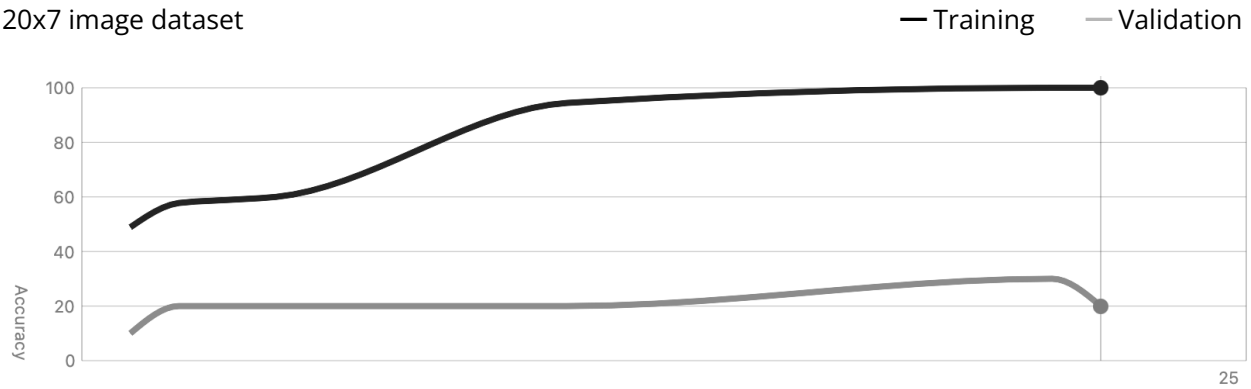
10x5 image dataset



18 iterations	Training	Validation	Test
Accuracy	100.0	60.0	24
Precision	100.0	40.0	23.0
Recall	100.0	60.0	60.0

The first dataset of 10 images per category showed promise. With test accuracy being above 20% this model was already better than chance, but there was still lots of room for improvement. The model failed to correctly identify any images as sadness in testing and had the highest recall and false positives in anger.

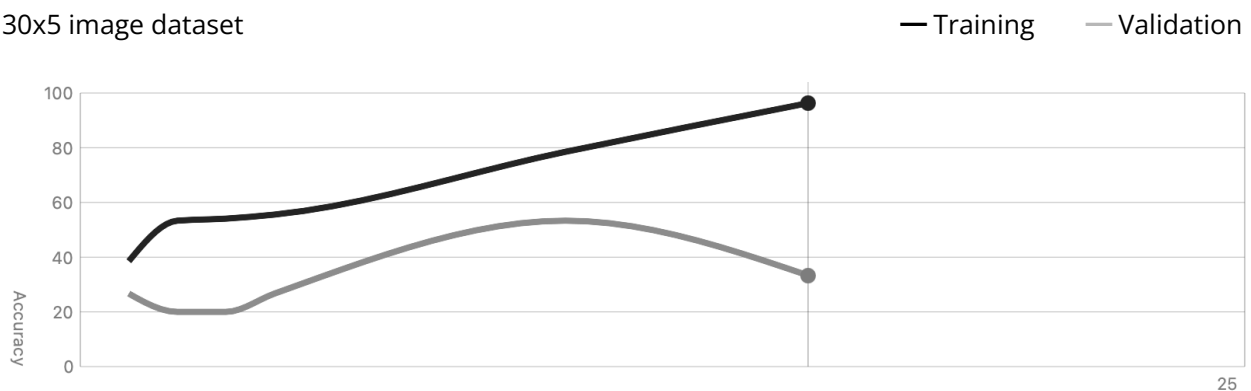
20x7 image dataset



22 iterations	Training	Validation	Test
Accuracy	100.0	20.0	29
Precision	100.0	25.0	33.4
Recall	100.0	20.0	29.0

From training, the second model of 20 images per category seemed to perform worse than the first model, but after evaluation, this model performed better in testing. There was a more even distribution of correct classifications between 5 and 7 correct in each category.

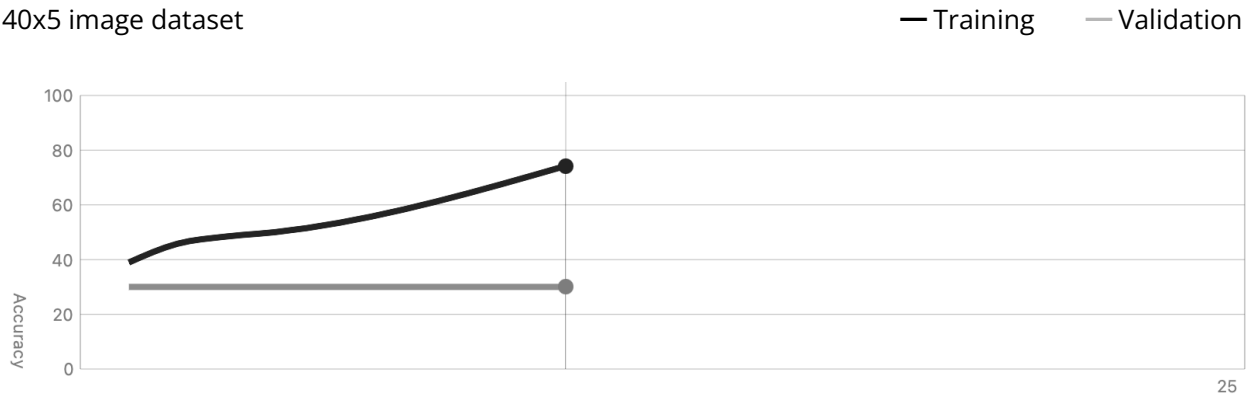
30x5 image dataset



16 iterations	Training	Validation	Test
Accuracy	96.3	33.3	34
Precision	96.4	29.8	35.2
Recall	96.4	33.2	34.0

The third dataset, with 30 images per category, followed the trend set by the second model. This model performed worse in training but better in testing than the previous models.

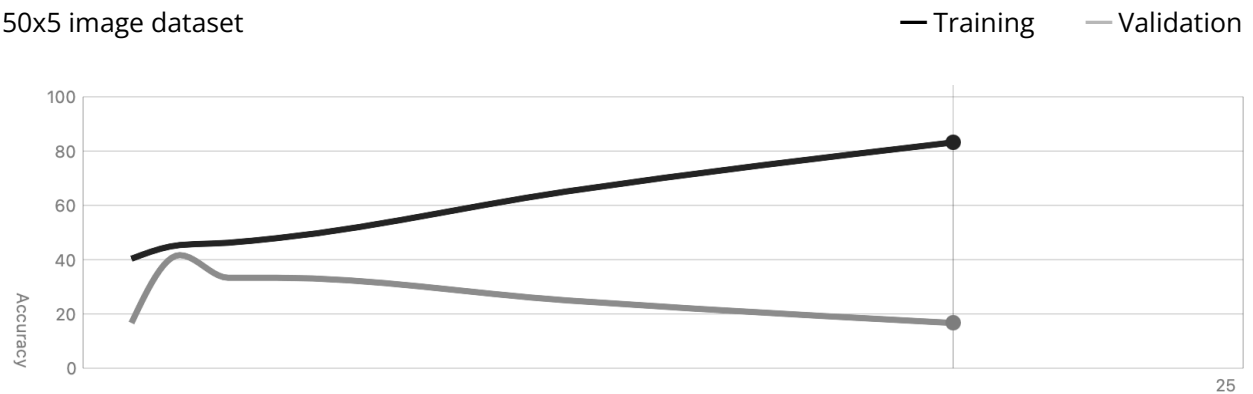
40x5 image dataset



11 iterations	Training	Validation	Test
Accuracy	74.2	30.0	27
Precision	75.6	16.6	25.4
Recall	74.4	30.0	27.0

The fourth dataset, with 40 images per category, broke the trend created by the previous two models. This model had some of the lowest values in iterations, training, and testing so far.

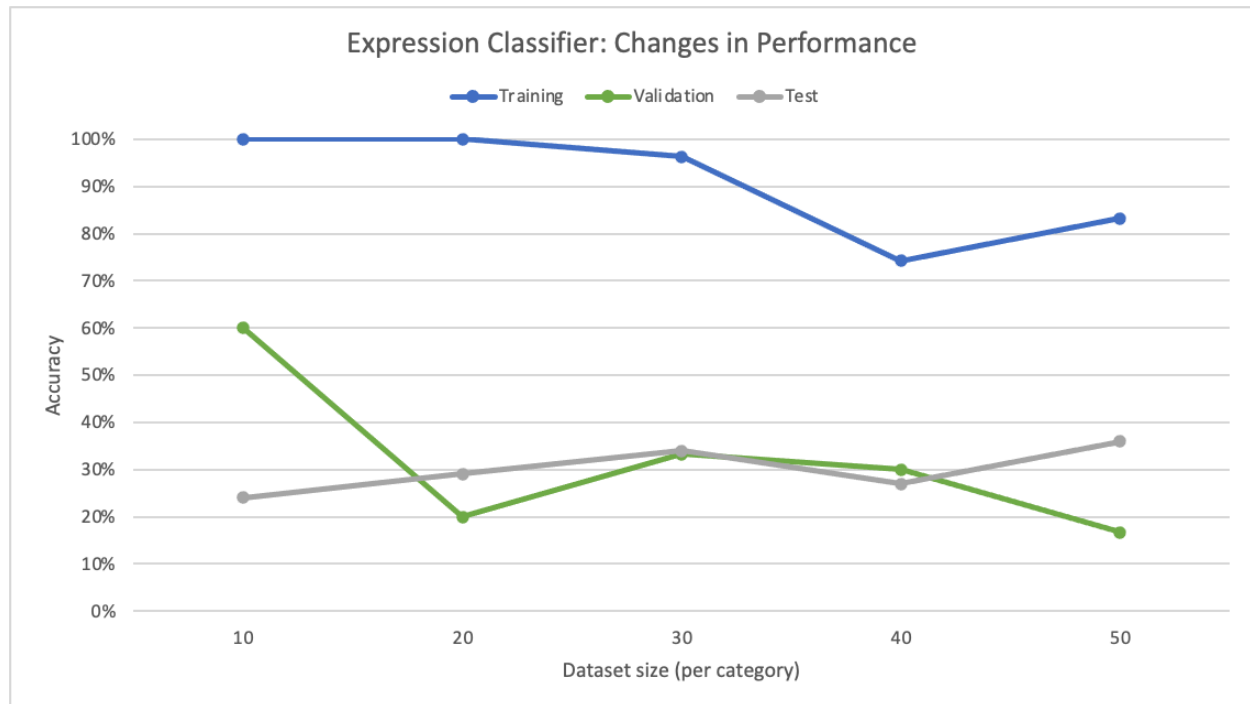
50x5 image dataset



19 iterations	Training	Validation	Test
Accuracy	83.2%	16.7%	36
Precision	83.2	20.0	36.4
Recall	83.4	16.6	36.0

The fifth and final dataset, with 50 images per category, showed the lowest performance in training but the best performance in testing

Changes in performance over different training conditions



Overall, as the dataset size increases, there is a downward trend in training and validation accuracy, while testing accuracy has a slight upward trend. This may indicate a correlation, but it is hard to say with so little information.

Results

The results indicate that facial expression recognition is a challenging task, even with controlled, small datasets. While all models achieved very high training accuracy, precision, and recall, their performance on validation and test sets was considerably lower, revealing significant overfitting. Increasing the number of images per category did not lead to a consistent improvement in test performance; for example, the 40x5 dataset performed worse than the 30x5 dataset. The best generalization occurred with the 50x5 dataset, suggesting that a larger, more varied dataset helps the model capture features better, but the gains are non-linear and dependent on the specific dataset composition. Precision and recall fluctuated across models, indicating that some expressions were systematically harder for the model to classify correctly.

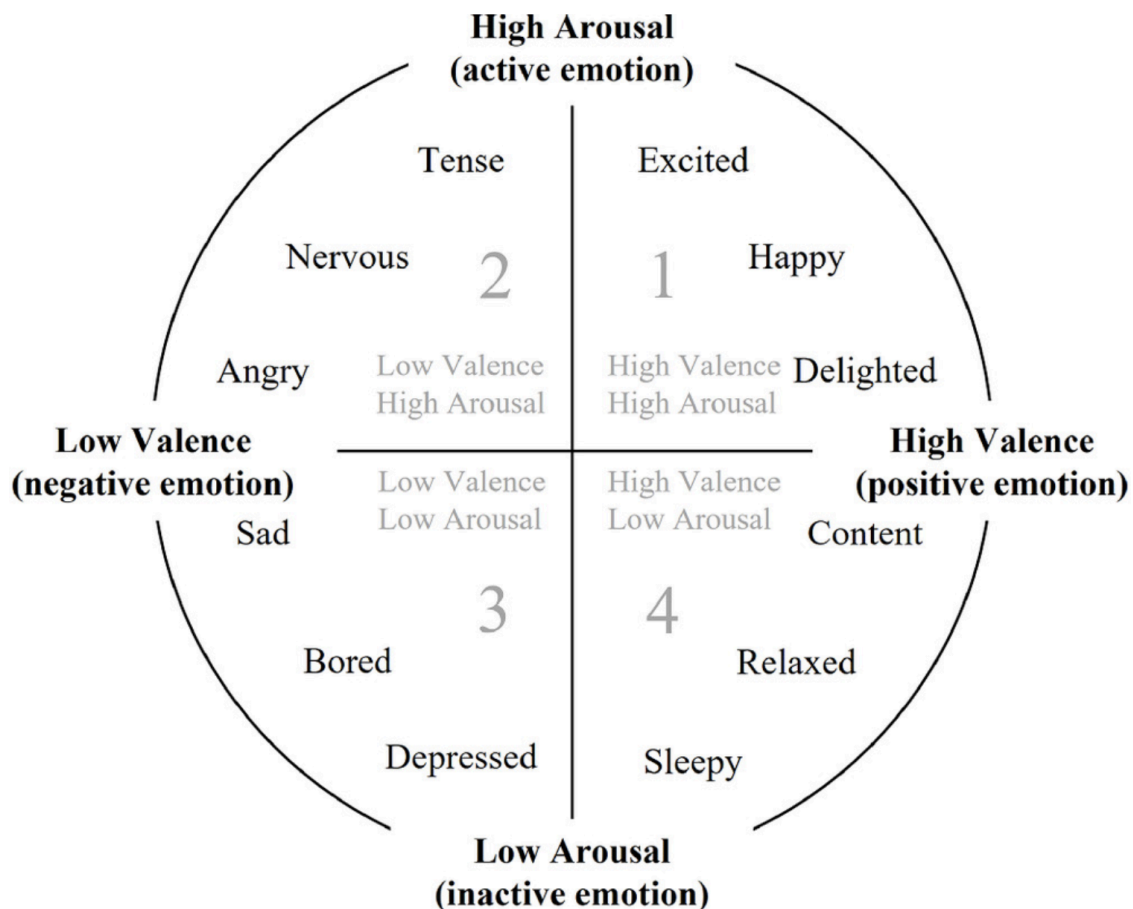
Based on the misclassifications observed, certain expressions are more difficult to recognize. Sadness proved particularly challenging, with some datasets failing to correctly identify any images in this category, while anger often had high false positives, meaning the model frequently misclassified other expressions as anger. These patterns suggest that negative expressions with subtler or overlapping facial cues are harder for the model to differentiate, compared to more distinct expressions like happiness. Overall, the results underscore that facial expression recognition

requires robust feature extraction and sufficient training data, highlighting the moderate-to-high difficulty of this task.

Additionally, untested factors such as incorrect labeling and image quality in the dataset may contribute to its performance. Using more data or even a different dataset, such as MIT's Affectiva or AffectNet, could yield better results. But both of those datasets are only shared upon request. For a more accurate model, I would suggest using a series of models to classify the expressions. Object recognition can be used to identify the face and recrop the input image to exclude miscellaneous surrounding data. Then, a combination of decision trees and classification can be employed to categorize the facial features more accurately.

Applications and Discussion

The facial expression classification model can be used to develop a creative application of symbolic audio generation. If we map the emotions on an XY grid of valence and arousal, we can potentially map the machine learning model to a Monte Carlo tree search. The different emotions identified in the XY graph created by Ferreira et al. can enable people to control music generated in real-time.



Resources:

Apple. (n.d.). *Create ML*. Apple Developer. <https://developer.apple.com/machine-learning/create-ml/>

Ferreira, L. N., Mou, L., Whitehead, J., & Lelis, L. H. S. (2022). Controlling perceived emotion in symbolic music generation with Monte Carlo tree search. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 18(1), 163–170.
<https://doi.org/10.1609/aiide.v18i1.21960>

Google. (n.d.). *Google AI Studio*. <https://aistudio.google.com/>

McDuff, D. J. (2013). *Affectiva-MIT facial expression dataset (AM-FED)* [Data set]. Massachusetts Institute of Technology. <https://dspace.mit.edu/handle/1721.1/80733>

Mollahosseini, A., Hasani, B., & Mahoor, M. H. (2017). *AffectNet: A new database for facial expression, valence, and arousal computation in the wild* [Data set]. *IEEE Transactions on Affective Computing*.
<https://doi.org/10.1109/TAFFC.2017.2740923>

muxspace. (n.d.). *facial_expressions* [GitHub repository]. GitHub.
https://github.com/muxspace/facial_expressions